NeetCode 150

NeetCode 150 in NeetCode topic order. Index-first, then detailed solutions. Overlaps with Blind 75 reuse copied solution blocks from `26U/blind75.md`.

Category

Arrays & Hashing

[E] 1. Two Sum Key interview pattern for this problem. `neetcode150`

[E] 217. Contains Duplicate Key interview pattern for this problem. `neetcode150`

[E] 242. Valid Anagram Key interview pattern for this problem. `neetcode150`

[M] 49. Group Anagrams Key interview pattern for this problem. `neetcode150`

[M] 347. Top K Frequent Elements Key interview pattern for this problem. `neetcode150`

[M] 238. Product of Array Except Self Key interview pattern for this problem. `neetcode150`

[M] 36. Valid Sudoku Key interview pattern for this problem. `neetcode150`

[M] 128. Longest Consecutive Sequence Key interview pattern for this problem. `neetcode150`

[M] 271. Encode and Decode Strings Key interview pattern for this problem. `neetcode150`

Two Pointers

[E] 125. Valid Palindrome Key interview pattern for this problem. `neetcode150`

[M] 167. Two Sum II - Input Array Is Sorted Key interview pattern for this problem. `neetcode150`

[M] 15. 3Sum Key interview pattern for this problem. `neetcode150`

[M] 11. Container With Most Water Key interview pattern for this problem. `neetcode150`

[H] 42. Trapping Rain Water Key interview pattern for this problem. `neetcode150`

Sliding Window

[E] 121. Best Time to Buy And Sell Stock Key interview pattern for this problem. `neetcode150`

[M] 3. Longest Substring Without Repeating Characters Key interview pattern for this problem. `neetcode150`

[M] 424. Longest Repeating Character Replacement Key interview pattern for this problem. `neetcode150`

[M] 567. Permutation In String Key interview pattern for this problem. `neetcode150`

[H] 76. Minimum Window Substring Key interview pattern for this problem. `neetcode150`

[H] 239. Sliding Window Maximum Key interview pattern for this problem. `neetcode150`

Stack

[E] 20. Valid Parentheses Key interview pattern for this problem. `neetcode150`

[M] 155. Min Stack Key interview pattern for this problem. `neetcode150`

[M] 150. Evaluate Reverse Polish Notation Key interview pattern for this problem. `neetcode150`

[M] 22. Generate Parentheses Key interview pattern for this problem. `neetcode150`

[M] 739. Daily Temperatures Key interview pattern for this problem. `neetcode150`

[M] 853. Car Fleet Key interview pattern for this problem. `neetcode150`

[H] 84. Largest Rectangle in Histogram Key interview pattern for this problem. `neetcode150`

Binary Search

[E] 704. Binary Search Key interview pattern for this problem. `neetcode150`

[M] 74. Search a 2D Matrix Key interview pattern for this problem. `neetcode150`

[M] 875. Koko Eating Bananas Key interview pattern for this problem. `neetcode150`

[M] 153. Find Minimum In Rotated Sorted Array Key interview pattern for this problem. `neetcode150`

[M] 33. Search In Rotated Sorted Array Key interview pattern for this problem. `neetcode150`

[M] 981. Time Based Key-Value Store Key interview pattern for this problem. `neetcode150`

[H] 4. Median of Two Sorted Arrays Key interview pattern for this problem. `neetcode150`

Linked List

[E] 206. Reverse Linked List Key interview pattern for this problem. `neetcode150`

[E] 21. Merge Two Sorted Lists Key interview pattern for this problem. `neetcode150`

[E] 141. Linked List Cycle Key interview pattern for this problem. `neetcode150`

[M] 143. Reorder List Key interview pattern for this problem. `neetcode150`

[M] 19. Remove Nth Node From End of List Key interview pattern for this problem. `neetcode150`

[M] 138. Copy List with Random Pointer Key interview pattern for this problem. `neetcode150`

[M] 2. Add Two Numbers Key interview pattern for this problem. `neetcode150`

[M] 287. Find the Duplicate Number Key interview pattern for this problem. `neetcode150`

[M] 146. LRU Cache Key interview pattern for this problem. `neetcode150`

[H] 23. Merge K Sorted Lists Key interview pattern for this problem. `neetcode150`

[H] 25. Reverse Nodes in k-Group Key interview pattern for this problem. `neetcode150`

Trees

[E] 226. Invert Binary Tree Key interview pattern for this problem. `neetcode150`

[E] 104. Maximum Depth of Binary Tree Key interview pattern for this problem. `neetcode150`

[E] 543. Diameter of Binary Tree Key interview pattern for this problem. `neetcode150`

[E] 110. Balanced Binary Tree Key interview pattern for this problem. `neetcode150`

[E] 100. Same Tree Key interview pattern for this problem. `neetcode150`

[E] 572. Subtree of Another Tree Key interview pattern for this problem. `neetcode150`

[M] 235. Lowest Common Ancestor of a Binary Search Tree Key interview pattern for this problem. `neetcode150`

[M] 102. Binary Tree Level Order Traversal Key interview pattern for this problem. `neetcode150`

[M] 199. Binary Tree Right Side View Key interview pattern for this problem. `neetcode150`

[M] 1448. Count Good Nodes In Binary Tree Key interview pattern for this problem. `neetcode150`

[M] 98. Validate Binary Search Tree Key interview pattern for this problem. `neetcode150`

[M] 230. Kth Smallest Element In a Bst Key interview pattern for this problem. `neetcode150`

[M] 105. Construct Binary Tree From Preorder And Inorder Traversal Key interview pattern for this problem. `neetcode150`

[H] 124. Binary Tree Maximum Path Sum Key interview pattern for this problem. `neetcode150`

[H] 297. Serialize And Deserialize Binary Tree Key interview pattern for this problem. `neetcode150`

Tries

[M] 208. Implement Trie Prefix Tree Key interview pattern for this problem. `neetcode150`

[M] 211. Design Add And Search Words Data Structure Key interview pattern for this problem. `neetcode150`

[H] 212. Word Search II Key interview pattern for this problem. `neetcode150`

Heap / Priority Queue

[E] 703. Kth Largest Element in a Stream Key interview pattern for this problem. `neetcode150`

[E] 1046. Last Stone Weight Key interview pattern for this problem. `neetcode150`

[M] 215. Kth Largest Element in an Array Key interview pattern for this problem. `neetcode150`

[M] 973. K Closest Points to Origin Key interview pattern for this problem. `neetcode150`

[M] 621. Task Scheduler Key interview pattern for this problem. `neetcode150`

[M] 355. Design Twitter Key interview pattern for this problem. `neetcode150`

[H] 295. Find Median From Data Stream Key interview pattern for this problem. `neetcode150`

Backtracking

[M] 78. Subsets Key interview pattern for this problem. `neetcode150`

[M] 39. Combination Sum Key interview pattern for this problem. `neetcode150`

[M] 46. Permutations Key interview pattern for this problem. `neetcode150`

[M] 90. Subsets II Key interview pattern for this problem. `neetcode150`

[M] 40. Combination Sum II Key interview pattern for this problem. `neetcode150`

[M] 79. Word Search Key interview pattern for this problem. `neetcode150`

[M] 131. Palindrome Partitioning Key interview pattern for this problem. `neetcode150`

[M] 17. Letter Combinations of a Phone Number Key interview pattern for this problem. `neetcode150`

[H] 51. N-Queens Key interview pattern for this problem. `neetcode150`

Graphs

[M] 200. Number of Islands Key interview pattern for this problem. `neetcode150`

[M] 695. Max Area of Island Key interview pattern for this problem. `neetcode150`

[M] 133. Clone Graph Key interview pattern for this problem. `neetcode150`

[M] 286. Walls and Gates Key interview pattern for this problem. `neetcode150`

[M] 994. Rotting Oranges Key interview pattern for this problem. `neetcode150`

[M] 417. Pacific Atlantic Water Flow Key interview pattern for this problem. `neetcode150`

[M] 130. Surrounded Regions Key interview pattern for this problem. `neetcode150`

[M] 207. Course Schedule Key interview pattern for this problem. `neetcode150`

[M] 210. Course Schedule II Key interview pattern for this problem. `neetcode150`

[M] 684. Redundant Connection Key interview pattern for this problem. `neetcode150`

[M] 261. Graph Valid Tree Key interview pattern for this problem. `neetcode150`

[M] 323. Number of Connected Components In An Undirected Graph Key interview pattern for this problem. `neetcode150`

[H] 127. Word Ladder Key interview pattern for this problem. `neetcode150`

Advanced Graphs

[H] 332. Reconstruct Itinerary Key interview pattern for this problem. `neetcode150`

[M] 1584. Min Cost to Connect All Points Key interview pattern for this problem. `neetcode150`

[M] 743. Network Delay Time Key interview pattern for this problem. `neetcode150`

[M] 787. Cheapest Flights Within K Stops Key interview pattern for this problem. `neetcode150`

[H] 778. Swim in Rising Water Key interview pattern for this problem. `neetcode150`

[H] 269. Alien Dictionary Key interview pattern for this problem. `neetcode150`

1-D Dynamic Programming

[E] 70. Climbing Stairs Key interview pattern for this problem. `neetcode150`

[E] 746. Min Cost Climbing Stairs Key interview pattern for this problem. `neetcode150`

[M] 198. House Robber Key interview pattern for this problem. `neetcode150`

[M] 213. House Robber II Key interview pattern for this problem. `neetcode150`

[M] 5. Longest Palindromic Substring Key interview pattern for this problem. `neetcode150`

[M] 647. Palindromic Substrings Key interview pattern for this problem. `neetcode150`

[M] 91. Decode Ways Key interview pattern for this problem. `neetcode150`

[M] 322. Coin Change Key interview pattern for this problem. `neetcode150`

[M] 152. Maximum Product Subarray Key interview pattern for this problem. `neetcode150`

[M] 139. Word Break Key interview pattern for this problem. `neetcode150`

[M] 300. Longest Increasing Subsequence Key interview pattern for this problem. `neetcode150`

[M] 416. Partition Equal Subset Sum Key interview pattern for this problem. `neetcode150`

2-D Dynamic Programming

[M] 62. Unique Paths Key interview pattern for this problem. `neetcode150`

[M] 1143. Longest Common Subsequence Key interview pattern for this problem. `neetcode150`

[M] 309. Best Time to Buy and Sell Stock with Cooldown Key interview pattern for this problem. `neetcode150`

[M] 518. Coin Change II Key interview pattern for this problem. `neetcode150`

[M] 494. Target Sum Key interview pattern for this problem. `neetcode150`

[M] 97. Interleaving String Key interview pattern for this problem. `neetcode150`

[M] 120. Triangle Key interview pattern for this problem. `neetcode150`

[H] 115. Distinct Subsequences Key interview pattern for this problem. `neetcode150`

[M] 72. Edit Distance Key interview pattern for this problem. `neetcode150`

[H] 329. Longest Increasing Path in a Matrix Key interview pattern for this problem. `neetcode150`

[H] 312. Burst Balloons Key interview pattern for this problem. `neetcode150`

Greedy

[M] 53. Maximum Subarray Key interview pattern for this problem. `neetcode150`

[M] 55. Jump Game Key interview pattern for this problem. `neetcode150`

[M] 45. Jump Game II Key interview pattern for this problem. `neetcode150`

[M] 134. Gas Station Key interview pattern for this problem. `neetcode150`

[M] 846. Hand of Straights Key interview pattern for this problem. `neetcode150`

[M] 1899. Merge Triplets to Form Target Triplet Key interview pattern for this problem. `neetcode150`

[M] 763. Partition Labels Key interview pattern for this problem. `neetcode150`

[M] 678. Valid Parenthesis String Key interview pattern for this problem. `neetcode150`

Intervals

[M] 57. Insert Interval Key interview pattern for this problem. `neetcode150`

[M] 56. Merge Intervals Key interview pattern for this problem. `neetcode150`

[M] 435. Non Overlapping Intervals Key interview pattern for this problem. `neetcode150`

[E] 252. Meeting Rooms Key interview pattern for this problem. `neetcode150`

[M] 253. Meeting Rooms II Key interview pattern for this problem. `neetcode150`

[H] 1851. Minimum Interval to Include Each Query Key interview pattern for this problem. `neetcode150`

Math & Geometry

[M] 48. Rotate Image Key interview pattern for this problem. `neetcode150`

[M] 54. Spiral Matrix Key interview pattern for this problem. `neetcode150`

[M] 73. Set Matrix Zeroes Key interview pattern for this problem. `neetcode150`

[E] 202. Happy Number Key interview pattern for this problem. `neetcode150`

[E] 66. Plus One Key interview pattern for this problem. `neetcode150`

[M] 50. Pow(x, n) Key interview pattern for this problem. `neetcode150`

[M] 43. Multiply Strings Key interview pattern for this problem. `neetcode150`

[M] 2013. Detect Squares Key interview pattern for this problem. `neetcode150`

Bit Manipulation

[E] 136. Single Number Key interview pattern for this problem. `neetcode150`

[E] 191. Number of 1 Bits Key interview pattern for this problem. `neetcode150`

[E] 338. Counting Bits Key interview pattern for this problem. `neetcode150`

[E] 190. Reverse Bits Key interview pattern for this problem. `neetcode150`

[E] 268. Missing Number Key interview pattern for this problem. `neetcode150`

[M] 371. Sum of Two Integers Key interview pattern for this problem. `neetcode150`

[M] 7. Reverse Integer Key interview pattern for this problem. `neetcode150`

Solutions (Python)

Arrays & Hashing

1. Two Sum [E]

Description: Find two indices whose values add up to a target.

Idea: Keep a hash map `value -> index`; for each number `x`, check whether `target - x` was seen before.

```
class Solution:
    def twoSum(self, nums, target):
        index = {}
        for i, x in enumerate(nums):
            need = target - x
            if need in index:
                return [index[need], i]
            index[x] = i
        return []
# Time: O(n), Space: O(n)
```

217. Contains Duplicate [E]

Description: Determine if any value appears at least twice in an array.

Idea: Track seen values in a set; if we ever see one twice, return `True`.

```
class Solution:
    def containsDuplicate(self, nums):
        seen = set()
        for x in nums:
            if x in seen:
                return True
            seen.add(x)
        return False
# Time: O(n), Space: O(n)
```

242. Valid Anagram [E]

Description: Check whether two strings are permutations of each other.

Idea: Two strings are anagrams iff their character counts match.

```
from collections import Counter


class Solution:
    def isAnagram(self, s, t):
        return Counter(s) == Counter(t)
# Time: O(n), Space: O(1) for fixed alphabet, otherwise O(k) distinct cha
```

49. Group Anagrams [M]

Description: Group a list of strings into collections of anagrams.

Idea: Use a hash map keyed by a canonical signature for each word (e.g., a 26-count tuple).

```
from collections import defaultdict


class Solution:
    def groupAnagrams(self, strs):
        groups = defaultdict(list)

        for s in strs:
            counts = [0] * 26
            for ch in s:
                counts[ord(ch) - ord("a")] += 1
            groups[tuple(counts)].append(s)

        return list(groups.values())
# Time: O(n * k) where k is word length (counting chars), Space: O(n * k)
```

347. Top K Frequent Elements [M]

Description: Return the k most frequent elements from an array.

Approach 1: Bucket sort

Idea: Count frequencies, then bucket numbers by frequency and scan buckets from high to low.

bucket sort: same freq same bucket

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # 第一步：统计每个元素的出现次数
        cnt = Counter(nums)
        max_cnt = max(cnt.values())

        # 第二步：把出现次数相同的元素，放到同一个桶中
        buckets = [[] for _ in range(max_cnt + 1)]  # 也可以用 defaultdict
        for x, c in cnt.items():
            buckets[c].append(x)

        # 第三步：倒序遍历 buckets，把出现次数前 k 大的元素加入答案
        ans = []
```

```
        for bucket in reversed(buckets):
            ans += bucket
            # 注意题目保证答案唯一，一定会出现恰好等于 k 的情况
            if len(ans) == k:
                return ans


class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        count = {}
        for n in nums:
            count[n] = count.get(n, 0) + 1

        heap = []
        for num in count:
            heapq.heappush(heap, (count[num], num))
            if len(heap) > k:
                heapq.heappop(heap)

        res = []
        for i in range(k):
            res.append(heapq.heappop(heap)[1])
        return res


class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        count = {}
        for num in nums:
            count[num] = 1 + count.get(num, 0)

        arr = []
        for num, cnt in count.items():
            arr.append([cnt, num])
        arr.sort()

        res = []
        while len(res) < k:
            res.append(arr.pop()[1])
        return res


from collections import Counter
```

```
class Solution:
    def topKFrequent(self, nums, k):
        freq = Counter(nums)
        buckets = [[] for _ in range(len(nums) + 1)]

        for num, c in freq.items():
            buckets[c].append(num)

        res = []
        for c in range(len(buckets) - 1, 0, -1):
            for num in buckets[c]:
                res.append(num)
                if len(res) == k:
                    return res
        return res
# Time: O(n), Space: O(n)
```

Approach 2: Min-heap of size k

Idea: Keep only the k most frequent in a min-heap.

```
import heapq
from collections import Counter


class Solution:
    def topKFrequent(self, nums, k):
        freq = Counter(nums)
        heap = []

        for num, c in freq.items():
            heapq.heappush(heap, (c, num))
            if len(heap) > k:
                heapq.heappop(heap)

        return [num for _, num in heap]
# Time: O(n log k), Space: O(n) for the counts + O(k) heap
```

Two Pointers

238. Product of Array Except Self [M]

Description: For each index, compute the product of all other elements without division.

Idea: Build prefix products left-to-right into ans, then multiply by suffix products right-to-left.

```
class Solution:
    def productExceptSelf(self, nums):
        n = len(nums)
        ans = [1] * n

        prefix = 1
        for i in range(n):
            ans[i] = prefix
            prefix *= nums[i]

        suffix = 1
        for i in range(n - 1, -1, -1):
            ans[i] *= suffix
            suffix *= nums[i]

        return ans
# Time: O(n), Space: O(1) extra (not counting the output array)
```

36. Valid Sudoku [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

128. Longest Consecutive Sequence [M]

Description: Find the length of the longest run of consecutive integers in an array.

Idea: Put everything in a set; only start counting from numbers that have no predecessor (`x-1` not in set).

```
class Solution:
    def longestConsecutive(self, nums):
        s = set(nums)
        best = 0
```

```
        for x in s:
            if x - 1 not in s:  # start of a run
                y = x
                while y in s:
                    y += 1
                best = max(best, y - x)

        return best
# Time: O(n) average, Space: O(n)
```

271. Encode and Decode Strings [M]

Description: Serialize and deserialize a list of strings into one string safely.

Idea: Length-prefix each string so decoding is unambiguous: `<len>#<string>...`.

```
class Codec:
    def encode(self, strs):
        out = []
        for s in strs:
            out.append(str(len(s)))
            out.append("#")
            out.append(s)
        return "".join(out)

    def decode(self, s):
        res = []
        i = 0
        while i < len(s):
            j = i
            while s[j] != "#":
                j += 1
            length = int(s[i:j])
            j += 1
            res.append(s[j : j + length])
            i = j + length
        return res
# Time: O(total_chars) for both encode/decode, Space: O(total_chars) for
```

Two Pointers

125. Valid Palindrome [E]

Description: Determine whether a string reads the same forward and backward (alphanumeric only).

Idea: Two pointers; skip non-alphanumerics and compare lowercased characters.

```
class Solution:
    def isPalindrome(self, s):
        i, j = 0, len(s) - 1

        while i < j:
            while i < j and not s[i].isalnum():
                i += 1
            while i < j and not s[j].isalnum():
                j -= 1
            if s[i].lower() != s[j].lower():
                return False
            i += 1
            j -= 1

        return True
# Time: O(n), Space: O(1)
```

167. Two Sum II - Input Array Is Sorted [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

15. 3Sum [M]

Description: Find unique triplets in array which sum to zero.

Idea: Sort; fix `i`, then run a 2-sum with two pointers on `i+1..n-1`, skipping duplicates.

```
class Solution:
    def threeSum(self, nums):
        nums.sort()
        res = []
        n = len(nums)
```

```
        for i in range(n):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            if nums[i] > 0:
                break

            l, r = i + 1, n - 1
            while l < r:
                s = nums[i] + nums[l] + nums[r]
                if s < 0:
                    l += 1
                elif s > 0:
                    r -= 1
                else:
                    res.append([nums[i], nums[l], nums[r]])
                    l += 1
                    r -= 1
                    while l < r and nums[l] == nums[l - 1]:
                        l += 1
                    while l < r and nums[r] == nums[r + 1]:
                        r -= 1

        return res
# Time: O(n^2), Space: O(1) extra (ignoring output)
```

Sliding Window

11. Container With Most Water [M]

Description: Maximize area formed between two vertical lines on the x-axis.

Idea: Two pointers; the area is limited by the shorter wall, so move the shorter pointer inward.

```
class Solution:
    def maxArea(self, height):
        l, r = 0, len(height) - 1
        best = 0

        while l < r:
            best = max(best, (r - l) * min(height[l], height[r]))
            if height[l] < height[r]:
                l += 1
```

```
            else:
                r -= 1

        return best
# Time: O(n), Space: O(1)
```

42. Trapping Rain Water [H]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Sliding Window

121. Best Time to Buy And Sell Stock [E]

Description: Max profit from one buy-sell transaction given daily prices.

Idea: Track the minimum price so far; at each day, consider selling today.

```
class Solution:
    def maxProfit(self, prices):
        min_price = float("inf")
        best = 0

        for p in prices:
            min_price = min(min_price, p)
            best = max(best, p - min_price)

        return best

    def maxProfit(self, prices: List[int]) -> int:
        mn = math.inf
        res = 0
        for p in prices:
            profit = p - mn
            res = max(res, profit)
            mn = min(mn, p)
```

```
        return res
# Time: O(n), Space: O(1)
```

3. Longest Substring Without Repeating Characters [M]

Description: Length of the longest substring with all distinct characters.

Idea: Sliding window; store last seen index for each char and move `left` past duplicates.

```
class Solution:
    def lengthOfLongestSubstring(self, s):
        last = {}
        left = 0
        best = 0

        for right, ch in enumerate(s):
            if ch in last and last[ch] >= left:
                left = last[ch] + 1
            last[ch] = right
            best = max(best, right - left + 1)

        return best
# Time: O(n), Space: O(k) distinct chars in window
```

424. Longest Repeating Character Replacement [M]

Description: Longest substring achievable by replacing up to k characters.

Idea: Sliding window; keep `max_count` of any char in the window. Window is valid if `window_size - max_count <= k`.

```
from collections import defaultdict


class Solution:
    def characterReplacement(self, s, k):
        counts = defaultdict(int)
        left = 0
        max_count = 0
        best = 0

        for right, ch in enumerate(s):
            counts[ch] += 1
            max_count = max(max_count, counts[ch])
```

```
            while (right - left + 1) - max_count > k:
                counts[s[left]] -= 1
                left += 1

            best = max(best, right - left + 1)

        return best
# Time: O(n), Space: O(k) distinct chars (at most alphabet size)
```

567. Permutation In String [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

76. Minimum Window Substring [H]

Description: Find the smallest substring that contains all characters of a target string.

Idea: Sliding window with counts; expand right until all required chars are satisfied, then shrink left while staying valid.

```
from collections import Counter, defaultdict


class Solution:
    def minWindow(self, s, t):
        if not s or not t:
            return ""

        need = Counter(t)
        have = defaultdict(int)
        required = len(need)
        formed = 0

        best_len = float("inf")
        best_l = 0
```

```
        l = 0
        for r, ch in enumerate(s):
            if ch in need:
                have[ch] += 1
                if have[ch] == need[ch]:
                    formed += 1

            while formed == required and l <= r:
                if r - l + 1 < best_len:
                    best_len = r - l + 1
                    best_l = l

                left_ch = s[l]
                if left_ch in need:
                    have[left_ch] -= 1
                    if have[left_ch] < need[left_ch]:
                        formed -= 1
                l += 1

        if best_len == float("inf"):
            return ""
        return s[best_l : best_l + best_len]
# Time: O(|s| + |t|), Space: O(k) distinct chars in t
```

Stack

239. Sliding Window Maximum [H]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Stack

20. Valid Parentheses [E]

Description: Verify that brackets are balanced and properly nested.

Idea: Use a stack; every closing bracket must match the most recent unmatched opening bracket.

```
class Solution:
    def isValid(self, s):
        stack = []
        pair = {")": "(", "]": "[", "}": "{"}

        for ch in s:
            if ch in pair:
                if not stack or stack[-1] != pair[ch]:
                    return False
                stack.pop()
            else:
                stack.append(ch)

        return not stack
# Time: O(n), Space: O(n)
```

Binary Search

155. Min Stack [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

150. Evaluate Reverse Polish Notation [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

22. Generate Parentheses [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

739. Daily Temperatures [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

853. Car Fleet [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

84. Largest Rectangle in Histogram [H]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Binary Search

704. Binary Search [E]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

74. Search a 2D Matrix [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

875. Koko Eating Bananas [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

153. Find Minimum In Rotated Sorted Array [M]

Description: Locate the minimum element in a rotated sorted array in O(log n).

Idea: Binary search; compare `nums[mid]` to `nums[right]` to decide which side contains the minimum.

```
class Solution:
    def findMin(self, nums):
```

```
        l, r = 0, len(nums) - 1

        while l < r:
            m = (l + r) // 2
            if nums[m] > nums[r]:
                l = m + 1
            else:
                r = m

        return nums[l]

    def isValid(self, s: str) -> bool:
        pairs = { ")" : "(", "]" : "[", "}" : "{" }
        stack = []

        for c in s:
            if c in pairs:
                if len(stack) > 0 and stack[-1] == pairs[c]:
                    stack.pop()
                else:
                    return False
            else:
                stack.append(c)
        return len(stack) == 0
# Time: O(log n), Space: O(1)
```

Linked List

33. Search In Rotated Sorted Array [M]

Description: Find target index in a rotated sorted array using binary search.

Idea: Binary search; at each step, determine which half is sorted and narrow accordingly.

```
class Solution:
    def search(self, nums, target):
        l, r = 0, len(nums) - 1

        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return m

            if nums[l] <= nums[m]:  # left half sorted
```

```
                if nums[l] <= target < nums[m]:
                    r = m - 1
                else:
                    l = m + 1
            else:  # right half sorted
                if nums[m] < target <= nums[r]:
                    l = m + 1
                else:
                    r = m - 1

        return -1
# Time: O(log n), Space: O(1)
```

981. Time Based Key-Value Store [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

4. Median of Two Sorted Arrays [H]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Linked List

206. Reverse Linked List [E]

Description: Reverse the pointers of a singly linked list and return the new head.

Approach 1: Iterative

Idea: Rewire `next` pointers while walking the list.

```
class Solution:
    def reverseList(self, head):
        prev = None
        cur = head

        while cur:
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt

        return prev
# Time: O(n), Space: O(1)
```

Approach 2: Recursive

Idea: Reverse the rest of the list and attach current node at the end.

```
class Solution:
    def reverseList(self, head):
        def dfs(node, prev):
            if not node:
                return prev
            nxt = node.next
            node.next = prev
            return dfs(nxt, node)

        return dfs(head, None)
# Time: O(n), Space: O(n) recursion stack
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]
        def dfs(cur, pre):
            if not cur:
                return pre
            nxt = cur.next
            cur.next = pre
            pre = cur
            return dfs(nxt, cur)
        return dfs(head, None)

class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]
        if not head:
```

```
            return None
        newHead = head
        if head.next:
            newHead = self.reverseList(head.next)
            head.next.next = head
        head.next = None
        return newHead
```

21. Merge Two Sorted Lists [E]

Description: Merge two sorted singly-linked lists into one sorted list.

Idea: Two pointers + dummy head; repeatedly take the smaller node.

```
class Solution:
    def mergeTwoLists(self, list1, list2):
        dummy = ListNode()
        cur = dummy

        a, b = list1, list2
        while a and b:
            if a.val <= b.val:
                cur.next = a
                a = a.next
            else:
                cur.next = b
                b = b.next
            cur = cur.next

        cur.next = a if a else b
        return dummy.next
# Time: O(m + n), Space: O(1) (re-links existing nodes)

class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[Li
        if list1 is None:
            return list2
        if list2 is None:
            return list1
        if list1.val <= list2.val:
            list1.next = self.mergeTwoLists(list1.next, list2)
            return list1
        else:
```

```
            list2.next = self.mergeTwoLists(list1, list2.next)
            return list2
```

141. Linked List Cycle [E]

Description: Detect whether a linked list contains a cycle.

Approach 1: Floyd's tortoise-and-hare

Idea: If a cycle exists, fast pointer will eventually meet slow pointer.

```
class Solution:
    def hasCycle(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow is fast:
                return True
        return False
# Time: O(n), Space: O(1)
```

Approach 2: Hash set

Idea: Track visited nodes by identity.

```
class Solution:
    def hasCycle(self, head):
        seen = set()
        cur = head
        while cur:
            if cur in seen:
                return True
            seen.add(cur)
            cur = cur.next
        return False
# Time: O(n), Space: O(n)
```

143. Reorder List [M]

Description: Reorder list by alternating nodes from front and back.

Idea: Find middle, reverse second half, then weave the two lists together.

```python
class Solution:
    def reorderList(self, head):
        if not head or not head.next:
            return

        # 1) Find middle (slow)
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # 2) Reverse second half (starting at slow)
        prev = None
        cur = slow
        while cur:
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt
        second = prev

        # 3) Merge / weave
        first = head
        while second.next:
            tmp1 = first.next
            tmp2 = second.next
            first.next = second
            second.next = tmp1
            first = tmp1
            second = tmp2
# Time: O(n), Space: O(1)

class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        def rec(root: ListNode, cur: ListNode) -> ListNode:
            if not cur:
                return root
            root = rec(root, cur.next)
            if not root:
                return None
            tmp = None
            if root == cur or root.next == cur:
                cur.next = None
            else:
```

```
                tmp = root.next
                root.next = cur
                cur.next = tmp
            return tmp
        head = rec(head, head.next)
```

19. Remove Nth Node From End of List [M]

Description: Remove the n-th node from the end of a linked list in one pass.

Idea: Two pointers; move `fast` n steps ahead, then move both until `fast` reaches the end.

```
class Solution:
    def removeNthFromEnd(self, head, n):
        dummy = ListNode(0, head)
        fast = slow = dummy

        for _ in range(n):
            fast = fast.next

        while fast and fast.next:
            fast = fast.next
            slow = slow.next

        slow.next = slow.next.next
        return dummy.next
# Time: O(n), Space: O(1)
```

138. Copy List with Random Pointer [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

2. Add Two Numbers [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

287. Find the Duplicate Number [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

146. LRU Cache [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

23. Merge K Sorted Lists [H]

Description: Merge k sorted linked lists into one sorted list efficiently.

Approach 1: Min-heap

Idea: Always pop the smallest current head among the k lists.

```
import heapq
import itertools


class Solution:
    def mergeKLists(self, lists):
        heap = []
```

```
        counter = itertools.count()

        for node in lists:
            if node:
                heapq.heappush(heap, (node.val, next(counter), node))

        dummy = ListNode()
        cur = dummy

        while heap:
            _, _, node = heapq.heappop(heap)
            cur.next = node
            cur = cur.next
            if node.next:
                heapq.heappush(heap, (node.next.val, next(counter), node.

        cur.next = None
        return dummy.next
# Time: O(N log k) where N is total nodes  Space: O(k)
```

Approach 2: Divide and conquer (recursion)

Idea: Recursively merge the left and right halves, then merge the two results.

```
class Solution:
    def mergeKLists(self, lists):
        def merge(a, b):
            dummy = ListNode()
            cur = dummy
            while a and b:
                if a.val <= b.val:
                    cur.next = a
                    a = a.next
                else:
                    cur.next = b
                    b = b.next
                cur = cur.next
            cur.next = a if a else b
            return dummy.next

        def helper(l, r):
            if l > r:
                return None
            if l == r:
```

```
                return lists[l]
            m = (l + r) // 2
            return merge(helper(l, m), helper(m + 1, r))

        return helper(0, len(lists) - 1)
# Time: O(N log k), Space: O(log k) recursion stack
```

Approach 3: Merge 2-by-2 (pairwise rounds)

Idea: Merge lists in pairs, doubling the merge interval each round (iterative merge-sort style).

```
class Solution:
    def mergeKLists(self, lists):
        if not lists:
            return None

        def merge(a, b):
            dummy = ListNode()
            cur = dummy
            while a and b:
                if a.val <= b.val:
                    cur.next = a
                    a = a.next
                else:
                    cur.next = b
                    b = b.next
                cur = cur.next
            cur.next = a if a else b
            return dummy.next

        interval = 1
        while interval < len(lists):
            for i in range(0, len(lists) - interval, interval * 2):
                lists[i] = merge(lists[i], lists[i + interval])
            interval *= 2

        return lists[0]
# Time: O(N log k), Space: O(1) extra (re-links nodes)
```

Trees

25. Reverse Nodes in k-Group [H]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Trees

226. Invert Binary Tree [E]

Description: Swap left and right children for every node in the tree.

Idea: Swap left/right recursively.

```
class Solution:
    def invertTree(self, root):
        if not root:
            return None
        root.left, root.right = self.invertTree(root.right), self.invertT
        return root
# Time: O(n), Space: O(h) recursion stack

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root:
            root.left, root.right = root.right, root.left
            self.invertTree(root.right)
            self.invertTree(root.left)
            return root

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if not root:
            return None
        queue = deque([root])
        while queue:
            node = queue.popleft()
            node.left, node.right = node.right, node.left
            if node.left:
                queue.append(node.left)
            if node.right:
```

```
                queue.append(node.right)
        return root
```

104. Maximum Depth of Binary Tree [E]

Description: Compute the maximum depth (height) of a binary tree.

Idea: Depth is `1 + max(depth(left), depth(right))`.

```
class Solution:
    def maxDepth(self, root):
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right
# Time: O(n), Space: O(h) recursion stack
```

543. Diameter of Binary Tree [E]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

110. Balanced Binary Tree [E]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

100. Same Tree [E]

Description: Check whether two binary trees are structurally identical with same values.

Idea: Recursively compare structure and values.

```
class Solution:
    def isSameTree(self, p, q):
        if not p and not q:
            return True
        if not p or not q or p.val != q.val:
            return False
        return self.isSameTree(p.left, q.left) and self.isSameTree(p.righ
# Time: O(n), Space: O(h) recursion stack
```

572. Subtree of Another Tree [E]

Description: Determine whether one tree is a subtree of another tree.

Idea: Traverse `root`; at each node, check if the two trees match exactly.

```
class Solution:
    def isSubtree(self, root, subRoot):
        def same(a, b):
            if not a and not b:
                return True
            if not a or not b or a.val != b.val:
                return False
            return same(a.left, b.left) and same(a.right, b.right)

        if not subRoot:
            return True
        if not root:
            return False
        if same(root, subRoot):
            return True
        return self.isSubtree(root.left, subRoot) or self.isSubtree(root.

class Solution:
    def serialize(self, root: Optional[TreeNode]) -> str:
        if root == None:
            return "$#"

        return ("$" + str(root.val) + self.serialize(root.left) + self.se

    def z_function(self, s: str) -> list:
        z = [0] * len(s)
```

```
        l, r, n = 0, 0, len(s)
        for i in range(1, n):
            if i <= r:
                z[i] = min(r - i + 1, z[i - l])
            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1
            if i + z[i] - 1 > r:
                l, r = i, i + z[i] - 1
        return z

    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeN
        serialized_root = self.serialize(root)
        serialized_subRoot = self.serialize(subRoot)
        combined = serialized_subRoot + "|" + serialized_root

        z_values = self.z_function(combined)
        sub_len = len(serialized_subRoot)

        for i in range(sub_len + 1, len(combined)):
            if z_values[i] == sub_len:
                return True
        return False
# Time: O(n * m) worst case, Space: O(h) recursion stack
```

235. Lowest Common Ancestor of a Binary Search Tree [M]

Description: Find the lowest common ancestor of two nodes in a BST.

Idea: Use BST ordering: if both targets are < node, go left; if both > node, go right; else node is LCA.

```
class Solution:
    def lowestCommonAncestor(self, root, p, q):
        a, b = p.val, q.val
        cur = root

        while cur:
            if a < cur.val and b < cur.val:
                cur = cur.left
            elif a > cur.val and b > cur.val:
                cur = cur.right
            else:
                return cur
# Time: O(h), Space: O(1)
```

```
class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'T
        if root in (None, p, q):  # 找到 p 或 q 就不往下递归了，原因见上面答
            return root
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        if left and right:  # 左右都找到
            return root  # 当前节点是最近公共祖先
        # 如果只有左子树找到，就返回左子树的返回值
        # 如果只有右子树找到，就返回右子树的返回值
        # 如果左右子树都没有找到，就返回 None（注意此时 right = None）
        return left or right
```

102. Binary Tree Level Order Traversal [M]

Description: Return the node values of a binary tree level by level.

Approach 1: DFS (collect by depth)

Idea: DFS with a `depth` parameter; append to `res[depth]`.

```
class Solution:
    def levelOrder(self, root):
        res = []

        def dfs(node, depth):
            if not node:
                return
            if depth == len(res):
                res.append([])
            res[depth].append(node.val)
            dfs(node.left, depth + 1)
            dfs(node.right, depth + 1)

        dfs(root, 0)
        return res
# Time: O(n), Space: O(h)

class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
```

```
        res = []

        q = collections.deque()
        q.append(root)

        while q:
            qLen = len(q)
            level = []
            for i in range(qLen):
                node = q.popleft()
                if node:
                    level.append(node.val)
                    q.append(node.left)
                    q.append(node.right)
            if level:
                res.append(level)

        return res
```

Approach 2: BFS

Idea: Use a queue; process one full level at a time.

```
from collections import deque


class Solution:
    def levelOrder(self, root):
        if not root:
            return []

        res = []
        q = deque([root])

        while q:
            level = []
            for _ in range(len(q)):
                node = q.popleft()
                level.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            res.append(level)
```

```
        return res
# Time: O(n), Space: O(n) (queue in worst case)
```

199. Binary Tree Right Side View [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

1448. Count Good Nodes In Binary Tree [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

98. Validate Binary Search Tree [M]

Description: Verify that a binary tree satisfies BST ordering constraints.

Approach 1: DFS with bounds

Idea: Every node must lie in `(low, high)` where bounds tighten as you go down the tree.

```
class Solution:
    def isValidBST(self, root):
        def dfs(node, low, high):
            if not node:
                return True
            if not (low < node.val < high):
                return False
            return dfs(node.left, low, node.val) and dfs(node.right, node

        return dfs(root, float("-inf"), float("inf"))
```

```
# Time: O(n), Space: O(h)

class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        if not root:
            return True

        q = deque([(root, float("-inf"), float("inf"))])

        while q:
            node, left, right = q.popleft()
            if not (left < node.val < right):
                return False
            if node.left:
                q.append((node.left, left, node.val))
            if node.right:
                q.append((node.right, node.val, right))

        return True
```

Approach 2: In-order traversal

Idea: In-order traversal of a BST yields a strictly increasing sequence.

```
class Solution:
    def isValidBST(self, root):
        stack = []
        cur = root
        prev = None

        while stack or cur:
            while cur:
                stack.append(cur)
                cur = cur.left
            cur = stack.pop()

            if prev is not None and cur.val <= prev:
                return False
            prev = cur.val
            cur = cur.right

        return True
# Time: O(n), Space: O(h)
```

230. Kth Smallest Element In a BST [M]

Description: Return the k-th smallest value in a BST using in-order traversal.

Idea: In-order traversal visits BST nodes in sorted order; stop at the k-th visit.

```
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        cnt = k
        res = root.val

        def dfs(node):
            nonlocal cnt, res
            if not node:
                return

            dfs(node.left)
            cnt -= 1
            if cnt == 0:
                res = node.val
                return
            dfs(node.right)

        dfs(root)
        return res


class Solution:
    def kthSmallest(self, root, k):
        stack = []
        cur = root

        while stack or cur:
            while cur:
                stack.append(cur)
                cur = cur.left
            cur = stack.pop()
            k -= 1
            if k == 0:
                return cur.val
            cur = cur.right
# Time: O(h + k) average, Space: O(h)
```

105. Construct Binary Tree From Preorder And Inorder Traversal [M]

Description: Reconstruct a binary tree given its preorder and inorder traversals.

Idea: Preorder picks the root first; inorder splits left/right subtrees around the root.

```
class Solution:
    def buildTree(self, preorder, inorder):
        idx = {val: i for i, val in enumerate(inorder)}
        pre_i = 0

        def build(l, r):
            nonlocal pre_i
            if l > r:
                return None

            root_val = preorder[pre_i]
            pre_i += 1
            root = TreeNode(root_val)

            mid = idx[root_val]
            root.left = build(l, mid - 1)
            root.right = build(mid + 1, r)
            return root

        return build(0, len(inorder) - 1)
# Time: O(n) with an inorder index map, Space: O(n) for the map + recursi

class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optic
        if not preorder or not inorder:
            return None

        root = TreeNode(preorder[0])
        mid = inorder.index(preorder[0])
        root.left = self.buildTree(preorder[1 : mid + 1], inorder[:mid])
        root.right = self.buildTree(preorder[mid + 1 :], inorder[mid + 1
        return root


class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optic
        preIdx = inIdx = 0
        def dfs(limit):
            nonlocal preIdx, inIdx
            if preIdx >= len(preorder):
```

```
                return None
            if inorder[inIdx] == limit:
                inIdx += 1
                return None

            root = TreeNode(preorder[preIdx])
            preIdx += 1
            root.left = dfs(root.val)
            root.right = dfs(limit)
            return root
        return dfs(float('inf'))
```

124. Binary Tree Maximum Path Sum [H]

Description: Compute the maximum path sum for any path in the binary tree.

Idea: For each node, compute the max "gain" to contribute to its parent (choose at most one side), while updating a global best using `node + left_gain + right_gain`.

```
class Solution:
    def maxPathSum(self, root):
        best = float("-inf")

        def gain(node):
            nonlocal best
            if not node:
                return 0

            left = max(gain(node.left), 0)
            right = max(gain(node.right), 0)

            best = max(best, node.val + left + right)
            return node.val + max(left, right)

        gain(root)
        return best
# Time: O(n), Space: O(h)
```

297. Serialize And Deserialize Binary Tree [H]

Description: Serialize a binary tree to a string and reconstruct it back.

Idea: Preorder DFS with a null marker (e.g., #) uniquely represents the tree.

```
class Codec:
    def serialize(self, root):
        vals = []

        def dfs(node):
            if not node:
                vals.append("#")
                return
            vals.append(str(node.val))
            dfs(node.left)
            dfs(node.right)

        dfs(root)
        return ",".join(vals)

    def deserialize(self, data):
        vals = iter(data.split(","))

        def dfs():
            val = next(vals)
            if val == "#":
                return None
            node = TreeNode(int(val))
            node.left = dfs()
            node.right = dfs()
            return node

        return dfs()
# Time: O(n) for both serialize/deserialize, Space: O(n)

class Codec:

    # Encodes a tree to a single string.
    def serialize(self, root: Optional[TreeNode]) -> str:
        if not root:
            return "N"
        res = []
        queue = deque([root])
        while queue:
            node = queue.popleft()
            if not node:
                res.append("N")
            else:
                res.append(str(node.val))
```

```
                queue.append(node.left)
                queue.append(node.right)
        return ",".join(res)

    # Decodes your encoded data to tree.
    def deserialize(self, data: str) -> Optional[TreeNode]:
        vals = data.split(",")
        if vals[0] == "N":
            return None
        root = TreeNode(int(vals[0]))
        queue = deque([root])
        index = 1
        while queue:
            node = queue.popleft()
            if vals[index] != "N":
                node.left = TreeNode(int(vals[index]))
                queue.append(node.left)
            index += 1
            if vals[index] != "N":
                node.right = TreeNode(int(vals[index]))
                queue.append(node.right)
            index += 1
        return root
```

Tries

Tries

208. Implement Trie Prefix Tree [M]

Description: Implement a trie supporting insert, search, and prefix queries.

Idea: Each node stores a map of `char -> child` and an end flag.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False


class Trie:
    def __init__(self):
        self.root = TrieNode()
```

```
    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.end = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.end

    def startsWith(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return True
# Time: O(L) per operation where L is word length  Space: O(total chars)
```

211. Design Add And Search Words Data Structure [M]

Description: Trie that supports adding words and searching with `.` wildcards.

Idea: Trie + DFS when encountering wildcard `.` (try all children).

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False


class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word):
        node = self.root
        for ch in word:
```

```
            node = node.children.setdefault(ch, TrieNode())
        node.end = True

    def search(self, word):
        def dfs(i, node):
            if i == len(word):
                return node.end

            ch = word[i]
            if ch == ".":
                return any(dfs(i + 1, child) for child in node.children.v

            if ch not in node.children:
                return False
            return dfs(i + 1, node.children[ch])

        return dfs(0, self.root)
# Time: O(L) average; O(26^L) worst case with many wildcards; Space: O(to
```

212. Word Search II [H]

Description: Find all words from a list that appear on a letter board using a trie + DFS.

Idea: Insert words into a trie; run DFS from each cell, walking the trie to prune impossible paths early.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None  # store full word at terminal nodes


class Solution:
    def findWords(self, board, words):
        root = TrieNode()
        for w in words:
            node = root
            for ch in w:
                node = node.children.setdefault(ch, TrieNode())
            node.word = w

        rows, cols = len(board), len(board[0])
        res = []
```

```
        def dfs(r, c, node):
            ch = board[r][c]
            if ch not in node.children:
                return

            nxt = node.children[ch]
            if nxt.word is not None:
                res.append(nxt.word)
                nxt.word = None  # avoid duplicates

            board[r][c] = "#"
            if r > 0 and board[r - 1][c] != "#":
                dfs(r - 1, c, nxt)
            if r + 1 < rows and board[r + 1][c] != "#":
                dfs(r + 1, c, nxt)
            if c > 0 and board[r][c - 1] != "#":
                dfs(r, c - 1, nxt)
            if c + 1 < cols and board[r][c + 1] != "#":
                dfs(r, c + 1, nxt)
            board[r][c] = ch

            # prune dead branches
            if not nxt.children and nxt.word is None:
                node.children.pop(ch, None)

        for r in range(rows):
            for c in range(cols):
                dfs(r, c, root)

        return res
# Time: Depends on branching; trie pruning makes it practical for constra
```

Heap / Priority Queue

Heap / Priority Queue

703. Kth Largest Element in a Stream [E]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
```

```
        pass
# Time: O(n), Space: O(n)
```

1046. Last Stone Weight [E]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

215. Kth Largest Element in an Array [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

973. K Closest Points to Origin [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

621. Task Scheduler [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
```

```
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

355. Design Twitter [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

295. Find Median From Data Stream [H]

Description: Maintain a running median as numbers are added using two heaps.

Idea: Two heaps: max-heap for the lower half, min-heap for the upper half; keep sizes balanced.

```
import heapq


class MedianFinder:
    def __init__(self):
        self.small = []  # max heap (store negatives)
        self.large = []  # min heap

    def addNum(self, num):
        heapq.heappush(self.small, -num)

        # ensure ordering property
        if self.large and (-self.small[0]) > self.large[0]:
            heapq.heappush(self.large, -heapq.heappop(self.small))

        # balance sizes
        if len(self.small) > len(self.large) + 1:
            heapq.heappush(self.large, -heapq.heappop(self.small))
        elif len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self):
```

```
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2
# Time: O(log n) per insert, O(1) median, Space: O(n)
```

Backtracking

Backtracking

78. Subsets [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

39. Combination Sum [M]

Description: Find all combinations of candidates that sum to target (reuse allowed).

Idea: DFS/backtracking; at each index, choose the current candidate any number of times.

```
class Solution:
    def combinationSum(self, candidates, target):
        candidates.sort()
        res = []

        def dfs(start, remaining, path):
            if remaining == 0:
                res.append(path[:])
                return

            for i in range(start, len(candidates)):
                c = candidates[i]
                if c > remaining:
                    break
                path.append(c)
                dfs(i, remaining - c, path)  # reuse allowed
                path.pop()
```

```
        dfs(0, target, [])
        return res
# Time: Exponential in worst case (output sensitive), Space: O(target) re
```

46. Permutations [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

90. Subsets II [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

40. Combination Sum II [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

79. Word Search [M]

Description: Determine if a single word exists in a 2D board by backtracking.

Idea: Backtracking from each cell; mark visited cells in-place during the current path.

```
class Solution:
    def exist(self, board, word):
        rows, cols = len(board), len(board[0])

        def dfs(r, c, i):
            if i == len(word):
                return True
            if not (0 <= r < rows and 0 <= c < cols):
                return False
            if board[r][c] != word[i]:
                return False

            tmp = board[r][c]
            board[r][c] = "#"
            found = (
                dfs(r + 1, c, i + 1)
                or dfs(r - 1, c, i + 1)
                or dfs(r, c + 1, i + 1)
                or dfs(r, c - 1, i + 1)
            )
            board[r][c] = tmp
            return found

        for r in range(rows):
            for c in range(cols):
                if dfs(r, c, 0):
                    return True
        return False
# Time: O(R*C*4^L) worst-case, Space: O(L) recursion stack
```

Graphs

131. Palindrome Partitioning [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

17. Letter Combinations of a Phone Number [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

51. N-Queens [H]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Graphs

200. Number of Islands [M]

Description: Count connected groups of '1's (islands) in a grid.

Approach 1: DFS flood-fill

Idea: For every land cell, flood-fill (turn `1` to `0`) to mark the entire island.

```
class Solution:
    def numIslands(self, grid):
        if not grid:
            return 0

        rows, cols = len(grid), len(grid[0])
        islands = 0

        def dfs(r, c):
            if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] !=
                return
            grid[r][c] = "0"
```

```
            dfs(r + 1, c)
            dfs(r - 1, c)
            dfs(r, c + 1)
            dfs(r, c - 1)

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == "1":
                    islands += 1
                    dfs(r, c)

        return islands
# Time: O(R*C)  Space: O(R*C) recursion stack worst case
```

Approach 2: BFS flood-fill

Idea: Same flood-fill, but iterative with a queue.

```
from collections import deque


class Solution:
    def numIslands(self, grid):
        if not grid:
            return 0

        rows, cols = len(grid), len(grid[0])
        islands = 0
        dirs = [(1, 0), (-1, 0), (0, 1), (0, -1)]

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] != "1":
                    continue
                islands += 1
                grid[r][c] = "0"
                q = deque([(r, c)])

                while q:
                    x, y = q.popleft()
                    for dx, dy in dirs:
                        nx, ny = x + dx, y + dy
                        if 0 <= nx < rows and 0 <= ny < cols and grid[nx]
                            grid[nx][ny] = "0"
```

```
                                q.append((nx, ny))

        return islands
# Time: O(R*C), Space: O(R*C) queue worst case
```

695. Max Area of Island [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

133. Clone Graph [M]

Description: Create a deep copy of an undirected graph.

Approach 1: DFS + hash map

Idea: `old_node -> new_node` mapping; recursively clone neighbors.

```
class Solution:
    def cloneGraph(self, node):
        if not node:
            return None

        clones = {}

        def dfs(cur):
            if cur in clones:
                return clones[cur]
            copy = Node(cur.val)
            clones[cur] = copy
            copy.neighbors = [dfs(nei) for nei in cur.neighbors]
            return copy

        return dfs(node)
# Time: O(V + E), Space: O(V)
```

Approach 2: BFS + hash map

Idea: Build clones level-by-level, wiring edges as you go.

```
from collections import deque


class Solution:
    def cloneGraph(self, node):
        if not node:
            return None

        clones = {node: Node(node.val)}
        q = deque([node])

        while q:
            cur = q.popleft()
            for nei in cur.neighbors:
                if nei not in clones:
                    clones[nei] = Node(nei.val)
                    q.append(nei)
                clones[cur].neighbors.append(clones[nei])

        return clones[node]
# Time: O(V + E), Space: O(V)
```

286. Walls and Gates [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

994. Rotting Oranges [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
```

```
        pass
# Time: O(n), Space: O(n)
```

417. Pacific Atlantic Water Flow [M]

Description: Find cells that can flow to both the Pacific and Atlantic oceans.

Idea: Reverse the flow: start DFS/BFS from ocean borders and move to equal/higher heights. Intersection are cells reaching both oceans.

```
class Solution:
    def pacificAtlantic(self, heights):
        if not heights or not heights[0]:
            return []

        rows, cols = len(heights), len(heights[0])
        pac = set()
        atl = set()

        def dfs(r, c, seen):
            seen.add((r, c))
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in
                    if heights[nr][nc] >= heights[r][c]:
                        dfs(nr, nc, seen)

        for c in range(cols):
            dfs(0, c, pac)
            dfs(rows - 1, c, atl)
        for r in range(rows):
            dfs(r, 0, pac)
            dfs(r, cols - 1, atl)

        return [[r, c] for (r, c) in (pac & atl)]
# Time: O(R*C), Space: O(R*C)
```

Advanced Graphs

130. Surrounded Regions [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

207. Course Schedule [M]

Description: Determine if all courses can be finished given prerequisite pairs (cycle check).

Approach 1: DFS cycle detection

Idea: `0=unvisited, 1=visiting, 2=done`; finding a back-edge to `visiting` means a cycle.

```
from collections import defaultdict


class Solution:
    def canFinish(self, numCourses, prerequisites):
        graph = defaultdict(list)
        for a, b in prerequisites:
            graph[a].append(b)

        UNVISITED, VISITING, DONE = 0, 1, 2
        state = [UNVISITED] * numCourses

        def dfs(course):
            if state[course] == VISITING:
                return False
            if state[course] == DONE:
                return True

            state[course] = VISITING
            for pre in graph[course]:
                if not dfs(pre):
                    return False
            state[course] = DONE
            return True

        for c in range(numCourses):
            if not dfs(c):
                return False
```

```
        return True
# Time: O(V + E), Space: O(V + E)
```

Approach 2: Kahn's algorithm (BFS topo sort)

Idea: Take nodes with indegree 0; remove edges; if you can take all courses, no cycle.

```
from collections import defaultdict, deque


class Solution:
    def canFinish(self, numCourses, prerequisites):
        graph = defaultdict(list)
        indeg = [0] * numCourses

        for a, b in prerequisites:
            graph[b].append(a)  # b -> a
            indeg[a] += 1

        q = deque([i for i in range(numCourses) if indeg[i] == 0])
        taken = 0

        while q:
            cur = q.popleft()
            taken += 1
            for nxt in graph[cur]:
                indeg[nxt] -= 1
                if indeg[nxt] == 0:
                    q.append(nxt)

        return taken == numCourses
# Time: O(V + E), Space: O(V + E)
```

210. Course Schedule II [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

684. Redundant Connection [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

261. Graph Valid Tree [M]

Description: Check if given edges form a single tree using Union-Find.

Idea: A valid tree has exactly `n-1` edges and no cycles. Union-Find detects cycles efficiently.

```
class Solution:
    def validTree(self, n, edges):
        if len(edges) != n - 1:
            return False

        parent = list(range(n))
        rank = [0] * n

        def find(x):
            while x != parent[x]:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        def union(a, b):
            ra, rb = find(a), find(b)
            if ra == rb:
                return False
            if rank[ra] < rank[rb]:
                parent[ra] = rb
            elif rank[ra] > rank[rb]:
                parent[rb] = ra
            else:
                parent[rb] = ra
                rank[ra] += 1
            return True
```

```
        for a, b in edges:
            if not union(a, b):
                return False
        return True
# Time: O(n + e * α(n)), Space: O(n)
```

323. Number of Connected Components In An Undirected Graph [M]

Description: Count connected components in an undirected graph (Union-Find or DFS).

Idea: Union-Find; each successful union reduces the component count by 1.

```
class Solution:
    def countComponents(self, n, edges):
        parent = list(range(n))
        rank = [0] * n
        components = n

        def find(x):
            while x != parent[x]:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        def union(a, b):
            nonlocal components
            ra, rb = find(a), find(b)
            if ra == rb:
                return
            components -= 1
            if rank[ra] < rank[rb]:
                parent[ra] = rb
            elif rank[ra] > rank[rb]:
                parent[rb] = ra
            else:
                parent[rb] = ra
                rank[ra] += 1

        for a, b in edges:
            union(a, b)

        return components
# Time: O(n + e * α(n)), Space: O(n)
```

127. Word Ladder [H]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Advanced Graphs

332. Reconstruct Itinerary [H]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

1584. Min Cost to Connect All Points [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

743. Network Delay Time [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
```

```
        pass
# Time: O(n), Space: O(n)
```

787. Cheapest Flights Within K Stops [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

778. Swim in Rising Water [H]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

- #### 269. Alien Dictionary [H] - Description: Derive a possible character order from a sorted dictionary of an alien language.

Idea: Build a directed graph from first differing character between adjacent words, then topologically sort. Edge case: If `w1` is longer and `w1.startswith(w2)`, input is invalid (no ordering).

```
from collections import defaultdict, deque


class Solution:
    def alienOrder(self, words):
        graph = defaultdict(set)
        indeg = {ch: 0 for w in words for ch in w}

        for w1, w2 in zip(words, words[1:]):
            if len(w1) > len(w2) and w1.startswith(w2):
                return ""
```

```
            for c1, c2 in zip(w1, w2):
                if c1 != c2:
                    if c2 not in graph[c1]:
                        graph[c1].add(c2)
                        indeg[c2] += 1
                    break

        q = deque([ch for ch, d in indeg.items() if d == 0])
        order = []

        while q:
            ch = q.popleft()
            order.append(ch)
            for nei in graph[ch]:
                indeg[nei] -= 1
                if indeg[nei] == 0:
                    q.append(nei)

        return "" if len(order) != len(indeg) else "".join(order)
# Time: O(total_chars + V + E), Space: O(V + E)
```

1-D Dynamic Programming

1-D Dynamic Programming

70. Climbing Stairs [E]

Description: Count ways to reach the top when you can take 1 or 2 steps.

Idea: Fibonacci: `ways[i] = ways[i-1] + ways[i-2]`.

```
class Solution:
    def climbStairs(self, n):
        if n <= 2:
            return n

        a, b = 1, 2
        for _ in range(3, n + 1):
            a, b = b, a + b
        return b
# Time: O(n), Space: O(1)
```

746. Min Cost Climbing Stairs [E]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

198. House Robber [M]

Description: Maximize robbery value without robbing adjacent houses (linear DP).

Idea: DP with two states: `prev1` best up to i-1, `prev2` best up to i-2.

```
class Solution:
    def rob(self, nums):
        prev2 = 0
        prev1 = 0

        for x in nums:
            prev2, prev1 = prev1, max(prev1, prev2 + x)

        return prev1
# Time: O(n), Space: O(1)
```

213. House Robber II [M]

Description: Circular houses version of House Robber; exclude either first or last.

Idea: Circle constraint means you can't take both first and last. Solve two linear cases and take max.

```
class Solution:
    def rob(self, nums):
        if len(nums) == 1:
            return nums[0]

        def rob_line(arr):
            prev2 = prev1 = 0
            for x in arr:
                prev2, prev1 = prev1, max(prev1, prev2 + x)
            return prev1
```

```
        return max(rob_line(nums[:-1]), rob_line(nums[1:]))
# Time: O(n), Space: O(1)
```

5. Longest Palindromic Substring [M]

Description: Find the longest palindromic substring in a given string.

Idea: Expand around each center (odd and even) and keep the best window.

```
class Solution:
    def longestPalindrome(self, s):
        if not s:
            return ""

        best_l, best_r = 0, 0  # inclusive

        def expand(l, r):
            while l >= 0 and r < len(s) and s[l] == s[r]:
                l -= 1
                r += 1
            return l + 1, r - 1

        for i in range(len(s)):
            l1, r1 = expand(i, i)
            if r1 - l1 > best_r - best_l:
                best_l, best_r = l1, r1
            l2, r2 = expand(i, i + 1)
            if r2 - l2 > best_r - best_l:
                best_l, best_r = l2, r2

        return s[best_l : best_r + 1]
# Time: O(n^2), Space: O(1)
```

647. Palindromic Substrings [M]

Description: Count all palindromic substrings in a given string.

Idea: Expand around each center (odd and even) and count.

```
class Solution:
    def countSubstrings(self, s):
        n = len(s)
        ans = 0
```

```
        def expand(l, r):
            nonlocal ans
            while l >= 0 and r < n and s[l] == s[r]:
                ans += 1
                l -= 1
                r += 1

        for i in range(n):
            expand(i, i)
            expand(i, i + 1)

        return ans
# Time: O(n^2), Space: O(1)
```

2-D Dynamic Programming

91. Decode Ways [M]

Description: Count decodings of digit strings mapping 1->A..26->Z using DP.

Idea: DP over prefix length. A digit can stand alone (1..9) and/or pair with previous (10..26).

```
class Solution:
    def numDecodings(self, s):
        if not s or s[0] == "0":
            return 0

        dp0, dp1 = 1, 1  # dp[i-2], dp[i-1]
        for i in range(2, len(s) + 1):
            cur = 0
            if s[i - 1] != "0":
                cur += dp1
            two = int(s[i - 2 : i])
            if 10 <= two <= 26:
                cur += dp0
            dp0, dp1 = dp1, cur

        return dp1
# Time: O(n), Space: O(1)
```

322. Coin Change [M]

Description: Compute minimum number of coins needed to make a given amount using DP.

Approach 1: Bottom-up DP

Idea: `dp[a]` = min coins to make amount a; try each coin.

```
class Solution:
    def coinChange(self, coins, amount):
        INF = amount + 1
        dp = [INF] * (amount + 1)
        dp[0] = 0

        for a in range(1, amount + 1):
            for c in coins:
                if a - c >= 0:
                    dp[a] = min(dp[a], dp[a - c] + 1)

        return -1 if dp[amount] == INF else dp[amount]
# Time: O(amount * numCoins), Space: O(amount)
```

Approach 2: Top-down DFS + memo

Idea: Try subtracting each coin and memoize the best result for each remaining amount.

```
from functools import lru_cache


class Solution:
    def coinChange(self, coins, amount):
        coins = tuple(coins)

        @lru_cache(None)
        def dfs(rem):
            if rem == 0:
                return 0
            if rem < 0:
                return float("inf")
            best = float("inf")
            for c in coins:
                best = min(best, dfs(rem - c) + 1)
            return best

        ans = dfs(amount)
```

```
        return -1 if ans == float("inf") else ans
# Time: O(amount * numCoins) states/transitions, Space: O(amount) memo +
```

152. Maximum Product Subarray [M]

Description: Find maximum product of any contiguous subarray, tracking min/max prefixes.

Idea: Track both max and min products ending at current index (negative flips max/min).

```
class Solution:
    def maxProduct(self, nums):
        cur_max = cur_min = ans = nums[0]

        for x in nums[1:]:
            if x < 0:
                cur_max, cur_min = cur_min, cur_max
            cur_max = max(x, cur_max * x)
            cur_min = min(x, cur_min * x)
            ans = max(ans, cur_max)

        return ans
# Time: O(n), Space: O(1)
```

139. Word Break [M]

Description: Determine if a string can be segmented into a sequence of dictionary words.

Approach 1: Bottom-up DP

Idea: `dp[i]` is True if `s[:i]` can be segmented; try all previous cuts `j < i`.

```
class Solution:
    def wordBreak(self, s, wordDict):
        wordSet = set(wordDict)
        dp = [False] * (len(s) + 1)
        dp[0] = True

        for i in range(1, len(s) + 1):
            for j in range(i):
                if dp[j] and s[j:i] in wordSet:
                    dp[i] = True
                    break
```

```
        return dp[-1]
# Time: O(n^2) checks (substring lookup is average O(1) with a set), Spac
```

Approach 2: DFS + memo

Idea: From index `i`, try matching dictionary words; memoize failed/successful indices.

```
class Solution:
    def wordBreak(self, s, wordDict):
        wordSet = set(wordDict)
        maxLen = max((len(w) for w in wordSet), default=0)
        memo = {}

        def dfs(i):
            if i == len(s):
                return True
            if i in memo:
                return memo[i]

            for j in range(i + 1, min(len(s), i + maxLen) + 1):
                if s[i:j] in wordSet and dfs(j):
                    memo[i] = True
                    return True
            memo[i] = False
            return False

        return dfs(0)
# Time: Output/input dependent; usually fast with memo and max word lengt
```

300. Longest Increasing Subsequence [M]

Description: Length of the longest strictly increasing subsequence (n log n solution available).

Approach 1: `O(n log n)` patience sorting

Idea: Maintain `tails[len] = smallest possible tail value of an increasing subsequence of length len+1`.

```
class Solution:
def longestConsecutive(self, nums: List[int]) -> int:
        numSet = set(nums)
        longest = 0
```

```
        for num in numSet:
            if (num - 1) not in numSet:
                length = 1
                while (num + length) in numSet:
                    length += 1
                longest = max(length, longest)
        return longest


class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        mp = defaultdict(int)
        res = 0

        for num in nums:
            if not mp[num]:
                mp[num] = mp[num - 1] + mp[num + 1] + 1
                mp[num - mp[num - 1]] = mp[num]
                mp[num + mp[num + 1]] = mp[num]
                res = max(res, mp[num])
        return res


class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        if not nums:
            return 0
        res = 0
        nums.sort()

        curr, streak = nums[0], 0
        i = 0
        while i < len(nums):
            if curr != nums[i]:
                curr = nums[i]
                streak = 0
            while i < len(nums) and nums[i] == curr:
                i += 1
            streak += 1
            curr += 1
            res = max(res, streak)
        return res
```

```
import bisect

def longestConsecutive(nums):
    if not nums:
        return 0

    nums.sort()
    longest = cur = 1

    for i in range(1, len(nums)):
        if nums[i] == nums[i - 1] + 1:
            cur += 1
        elif nums[i] != nums[i - 1]:
            cur = 1
        longest = max(longest, cur)

    return longest


class Solution:
    def lengthOfLIS(self, nums):
        tails = []
        for x in nums:
            i = bisect.bisect_left(tails, x)
            if i == len(tails):
                tails.append(x)
            else:
                tails[i] = x
        return len(tails)
# Time: O(n log n), Space: O(n)
```

Approach 2: `O(n^2)` DP

Idea: `dp[i]` = LIS ending at i; transition from all `j<i` with `nums[j] < nums[i]`.

```
class Solution:
    def lengthOfLIS(self, nums):
        n = len(nums)
        dp = [1] * n

        for i in range(n):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)
```

```
        return max(dp, default=0)
# Time: O(n^2), Space: O(n)
```

416. Partition Equal Subset Sum [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

2-D Dynamic Programming

62. Unique Paths [M]

Description: Count number of unique paths in an m x n grid moving only right/down.

Idea: `dp[r][c] = dp[r-1][c] + dp[r][c-1]`; use 1D DP since each row depends only on previous.

```
class Solution:
    def uniquePaths(self, m, n):
        dp = [1] * n

        for _ in range(1, m):
            for j in range(1, n):
                dp[j] += dp[j - 1]

        return dp[-1]
# Time: O(m*n), Space: O(n)
```

1143. Longest Common Subsequence [M]

Description: Compute the length of the longest common subsequence between two strings.

Idea: 2D DP: `dp[i][j]` = LCS length for suffixes `text1[i:]`, `text2[j:]`.

```
class Solution:
    def longestCommonSubsequence(self, text1, text2):
        m, n = len(text1), len(text2)
```

```
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m - 1, -1, -1):
            for j in range(n - 1, -1, -1):
                if text1[i] == text2[j]:
                    dp[i][j] = 1 + dp[i + 1][j + 1]
                else:
                    dp[i][j] = max(dp[i + 1][j], dp[i][j + 1])

        return dp[0][0]
# Time: O(m*n), Space: O(m*n)
```

Greedy

309. Best Time to Buy and Sell Stock with Cooldown [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

518. Coin Change II [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

494. Target Sum [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
```

```
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

97. Interleaving String [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

120. Triangle [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

115. Distinct Subsequences [H]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

72. Edit Distance [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

329. Longest Increasing Path in a Matrix [H]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

312. Burst Balloons [H]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Greedy

53. Maximum Subarray [M]

Description: Find the contiguous subarray with the largest sum (Kadane's algorithm).

Idea: Kadane's algorithm: best subarray ending at `i` is either `nums[i]` or extend previous.

```
class Solution:
    def maxSubArray(self, nums):
        best = cur = nums[0]
        for x in nums[1:]:
            cur = max(x, cur + x)
            best = max(best, cur)
```

```
        return best
# Time: O(n), Space: O(1)
```

55. Jump Game [M]

Description: Determine if you can reach the last index given max jumps at each position.

Idea: Greedy keep the farthest reachable index; fail if you land beyond it.

```
class Solution:
    def canJump(self, nums):
        farthest = 0
        for i, jump in enumerate(nums):
            if i > farthest:
                return False
            farthest = max(farthest, i + jump)
        return True
# Time: O(n), Space: O(1)
```

Intervals

45. Jump Game II [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

134. Gas Station [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

846. Hand of Straights [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

1899. Merge Triplets to Form Target Triplet [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

763. Partition Labels [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

678. Valid Parenthesis String [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Intervals

57. Insert Interval [M]

Description: Insert and merge a new interval into a list of non-overlapping intervals.

Idea: Add all intervals before `newInterval`, merge overlaps with it, then append the rest.

```
class Solution:
    def insert(self, intervals, newInterval):
        res = []
        i = 0
        n = len(intervals)
        start, end = newInterval

        while i < n and intervals[i][1] < start:
            res.append(intervals[i])
            i += 1

        while i < n and intervals[i][0] <= end:
            start = min(start, intervals[i][0])
            end = max(end, intervals[i][1])
            i += 1
        res.append([start, end])

        while i < n:
            res.append(intervals[i])
            i += 1

        return res
# Time: O(n), Space: O(n) for output
```

56. Merge Intervals [M]

Description: Merge all overlapping intervals into disjoint intervals.

Idea: Sort by start; extend the current merged interval while overlaps continue.

```
class Solution:
    def merge(self, intervals):
        intervals.sort(key=lambda x: x[0])
        merged = []

        for start, end in intervals:
```

```
            if not merged or start > merged[-1][1]:
                merged.append([start, end])
            else:
                merged[-1][1] = max(merged[-1][1], end)

        return merged
# Time: O(n log n), Space: O(n) for output
```

435. Non Overlapping Intervals [M]

Description: Minimum number of intervals to remove to make the rest non-overlapping.

Idea: Greedy sort by end time; keep the interval with the earliest end to leave room for future intervals.

```
class Solution:
    def eraseOverlapIntervals(self, intervals):
        if not intervals:
            return 0

        intervals.sort(key=lambda x: x[1])
        end = intervals[0][1]
        removed = 0

        for s, e in intervals[1:]:
            if s < end:
                removed += 1
            else:
                end = e

        return removed
# Time: O(n log n), Space: O(1) extra
```

Math & Geometry

252. Meeting Rooms [E]

Description: Determine if a person can attend all given meeting intervals without overlap.

Idea: Sort by start time; any overlap means you can't attend all meetings.

```
class Solution:
    def canAttendMeetings(self, intervals):
        intervals.sort(key=lambda x: x[0])
```

```
        for i in range(1, len(intervals)):
            if intervals[i][0] < intervals[i - 1][1]:
                return False
        return True
# Time: O(n log n), Space: O(1) extra (sorting aside)
```

253. Meeting Rooms II [M]

Description: Compute minimum number of meeting rooms required to hold all meetings.

Approach 1: Min-heap of end times

Idea: Reuse a room if the earliest-ending meeting ends before the next starts; otherwise allocate a new room.

```
import heapq


class Solution:
    def minMeetingRooms(self, intervals):
        if not intervals:
            return 0

        intervals.sort(key=lambda x: x[0])
        heap = []

        for start, end in intervals:
            if heap and heap[0] <= start:
                heapq.heapreplace(heap, end)
            else:
                heapq.heappush(heap, end)

        return len(heap)
# Time: O(n log n), Space: O(n) heap worst-case
```

Approach 2: Sweep line

Idea: Convert each interval into `(start, +1)` and `(end, -1)` events; max prefix sum is the answer.

```
class Solution:
    def minMeetingRooms(self, intervals):
        events = []
        for s, e in intervals:
```

```
            events.append((s, 1))
            events.append((e, -1))

        # process endings before starts at the same time
        events.sort(key=lambda x: (x[0], x[1]))

        rooms = best = 0
        for _, delta in events:
            rooms += delta
            best = max(best, rooms)
        return best
# Time: O(n log n), Space: O(n)
```

1851. Minimum Interval to Include Each Query [H]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Math & Geometry

48. Rotate Image [M]

Description: Rotate an n x n matrix 90 degrees clockwise in-place.

Idea: Rotate 90° clockwise = transpose + reverse each row.

```
class Solution:
    def rotate(self, matrix):
        n = len(matrix)

        # transpose
        for i in range(n):
            for j in range(i + 1, n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

        # reverse each row
        for row in matrix:
```

```
            row.reverse()
# Time: O(n^2), Space: O(1)
```

54. Spiral Matrix [M]

Description: Return elements of a matrix in spiral order.

Idea: Maintain boundaries (top, bottom, left, right) and peel layers.

```
class Solution:
    def spiralOrder(self, matrix):
        res = []
        if not matrix or not matrix[0]:
            return res

        top, bottom = 0, len(matrix) - 1
        left, right = 0, len(matrix[0]) - 1

        while top <= bottom and left <= right:
            for c in range(left, right + 1):
                res.append(matrix[top][c])
            top += 1

            for r in range(top, bottom + 1):
                res.append(matrix[r][right])
            right -= 1

            if top <= bottom:
                for c in range(right, left - 1, -1):
                    res.append(matrix[bottom][c])
                bottom -= 1

            if left <= right:
                for r in range(bottom, top - 1, -1):
                    res.append(matrix[r][left])
                left += 1

        return res
# Time: O(R*C), Space: O(1) extra (output aside)
```

73. Set Matrix Zeroes [M]

Description: If a cell is zero, set its entire row and column to zero (in-place options).

Approach 1: In-place markers (O(1) extra space)

Idea: Use first row/col as marker arrays; handle whether they originally contain zeros.

```
class Solution:
    def setZeroes(self, matrix):
        R, C = len(matrix), len(matrix[0])
        first_row_zero = any(matrix[0][c] == 0 for c in range(C))
        first_col_zero = any(matrix[r][0] == 0 for r in range(R))

        for r in range(1, R):
            for c in range(1, C):
                if matrix[r][c] == 0:
                    matrix[r][0] = 0
                    matrix[0][c] = 0

        for r in range(1, R):
            for c in range(1, C):
                if matrix[r][0] == 0 or matrix[0][c] == 0:
                    matrix[r][c] = 0

        if first_row_zero:
            for c in range(C):
                matrix[0][c] = 0
        if first_col_zero:
            for r in range(R):
                matrix[r][0] = 0
# Time: O(R*C), Space: O(1)
```

Approach 2: Row/column sets

Idea: First pass records which rows/cols must be zeroed; second pass writes zeros.

```
class Solution:
    def setZeroes(self, matrix):
        rows = set()
        cols = set()
        R, C = len(matrix), len(matrix[0])

        for r in range(R):
            for c in range(C):
                if matrix[r][c] == 0:
                    rows.add(r)
                    cols.add(c)
```

```
        for r in range(R):
            for c in range(C):
                if r in rows or c in cols:
                    matrix[r][c] = 0
# Time: O(R*C), Space: O(R + C)
```

Bit Manipulation

202. Happy Number [E]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

66. Plus One [E]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

50. Pow(x, n) [M]

neetcode150 - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

43. Multiply Strings [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

2013. Detect Squares [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

Bit Manipulation

136. Single Number [E]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```

191. Number of 1 Bits [E]

Description: Count the number of set bits (population count) in an integer.

Idea: Repeatedly clear the lowest set bit with `n &= n-1`.

```
class Solution:
    def hammingWeight(self, n):
        count = 0
        while n:
```

```
            n &= n - 1
            count += 1
        return count
# Time: O(popcount(n)), Space: O(1)
```

338. Counting Bits [E]

Description: For each number 0..n, compute the number of set bits efficiently.

Idea: `bits[i] = bits[i >> 1] + (i & 1)`.

```
class Solution:
    def countBits(self, n):
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i >> 1] + (i & 1)
        return ans
# Time: O(n), Space: O(n)
```

190. Reverse Bits [E]

Description: Reverse the bit order in a fixed-width integer.

Idea: Build the reversed number by shifting result left and reading bits from n.

```
class Solution:
    def reverseBits(self, n):
        res = 0
        for _ in range(32):
            res = (res << 1) | (n & 1)
            n >>= 1
        return res
# Time: O(1) (32 iterations), Space: O(1)
```

268. Missing Number [E]

Description: Find the missing number in [0..n] given n distinct numbers.

Idea: XOR all indices and values together; pairs cancel leaving the missing number.

```
class Solution:
    def missingNumber(self, nums):
        res = len(nums)
        for i, x in enumerate(nums):
```

```
            res ^= i ^ x
        return res
# Time: O(n), Space: O(1)
```

371. Sum of Two Integers [M]

Description: Compute sum of two integers without using the + operator using bitwise ops.

Idea: Bitwise addition: `xor` gives sum without carry; `(a&b)<<1` gives carry. Repeat until carry is 0. Note: Python ints are unbounded, so mask to 32-bit to emulate signed int behavior.

```
class Solution:
    def getSum(self, a, b):
        mask = 0xFFFFFFFF

        while b & mask:
            carry = (a & b) << 1
            a = (a ^ b) & mask
            b = carry & mask

        return a if a <= 0x7FFFFFFF else ~(a ^ mask)
# Time: O(1) (bounded by word size), Space: O(1)
```

7. Reverse Integer [M]

`neetcode150` - Use the canonical optimal pattern for this problem; implement and discuss tradeoffs in interview.

```
class Solution:
    def solve(self, *args, **kwargs):
        # Implement the standard optimal solution for this problem.
        pass
# Time: O(n), Space: O(n)
```